The Paved Road Had a Compliance Gate

Moving compliance from the end of the road into the road itself

Sumeet Ram
Senior Technical Product Manager

ILLUSTRATIVE

A new compliance rule can sit unused in a platform until a team goes looking

ALREADY INTEGRATED	A checkout team runs its market configuration through the platform's SDK
NEW RULE	Compliance adds a new data-handling rule for some markets
PLATFORM SHIPS IT	The platform ships support for it in the SDK, with release notes
WHAT THE TEAM SEES	Nothing. Their integration still works, so nobody looks
WHEN IT SURFACES	When compliance or an audit asks, and the scramble starts

SO WHAT

The platform had the answer. The team had no reason to ask for it, so it stayed in a silo until compliance came asking

SOURCE: illustrative scenario based on the speaker's platform experience

THE FAILURE

The geo-expansion platform already had the market rules, but teams only found them when compliance asked

1. Build the feature

Team ships for its home market first

→

2. Discover late

Market rules show up near the end of the sprint

→

3. File a request

Ticket to the platform team, then wait for guidance

→

4. Implement

Each team reads the SDK docs its own way

→

5. Ship with risk

Gaps surface in review, launch or audit

THE ROAD

A geo-expansion platform used by 150+ teams that lets a product launch in a new market by configuration

THE GAP

Platform capability shipped faster than teams could discover it. New rules reached product teams through release notes and tickets, so adoption was **triggered by compliance escalations**, rarely planned into the roadmap

SOURCE: speaker's platform experience

THE FAILURE

Bolting market logic on late produced five repeatable classes of failure

FAILURE CLASS	WHAT IT LOOKED LIKE
Version fragmentation	Teams stranded on old SDK versions while others moved on
Non-standard integration	The same requirement built five ways, and failing five ways
Late compliance gaps	Market requirements found after the code was written
Stalled launches	One missed market requirement holding a whole launch
Regulatory exposure	A market-specific requirement missed in a regulated flow

And the rules keep moving: regulators and payment networks publish new requirements market by market, on calendars that never match sprint plans. **A late-discovery process loses every one of these races.**

SOURCE: speaker's platform experience

WHAT EVERY PLATFORM TEAM HEARS

"We'll upgrade next quarter."

THE FAILURE

Every platform end-of-life becomes tech debt that consuming teams defer as long as they can

- An upgrade competes with roadmap work, so it slips to next quarter, then the one after
- The longer a team waits, the bigger and riskier the jump
- The platform ends up supporting several versions at once, and each one is another place to fail
- New compliance capabilities ship in the new version, so a deferred upgrade also defers the fix, like the new data-handling rule

For your platform: make the upgrade cheap enough that **deferring it stops making sense**

SOURCE: speaker's platform experience

THE FAILURE

ILLUSTRATIVE

The risk is a team's agent working off the road: confident, sometimes even correct, and ungoverned

	A TEAM'S AGENT, OFF THE ROAD	AN AGENT WORKING THROUGH THE PLATFORM
Context	Whatever the team researched	Everything the platform knows about that market
Rules	The agent's best reading of that research	Rules a custodian approved, with a version
Instructions	Nothing specific to markets	Platform instructions and guardrails in every answer
If it is wrong	Found at QA, launch or audit	Caught by a person before it ships
If it is right	Right once, then drifts as rules change	Stays right, because the platform keeps it current

SO WHAT

Give agents the platform's full context and a person's approval, and a wrong answer becomes **unlikely, and visible when it happens**

SOURCE: illustrative comparison based on the speaker's platform experience

Moving the gate into the road

Exposing the geo-expansion platform to agents through an MCP (Model Context Protocol) server, with people at the gate

THE NEW ROAD

Exposing the geo-expansion platform as an MCP server let agents find the rules at design and integrate from day one

STAGE	WHAT THE MCP SERVER DOES	TOOL KIND	WHAT IT REPLACES
Design	Reviews the PRD for missing market and compliance requirements	Adopt	Discovering market rules near the end of the sprint
Build	Generates market-configured integration code from the PRD	Adopt	Each team interpreting the SDK docs its own way
Maintain	Migrates code off deprecated versions and brings new rules to the flows they affect	Adopt, Update	Hand migrations deferred as tech debt; rules found at audit
Release	Checks output against current rules before launch, and keeps checking after	Verify	Gaps found in review, launch or audit

The platform lives in the IDE and coding agents developers already use. A new flow now starts by asking the platform, and a new rule like the data-handling one reaches every team through the same tools, **before compliance has to ask.**

SOURCE: speaker's platform; MCP = Model Context Protocol; PRD = product requirements document

THE NEW ROAD

Any platform can expose three kinds of tools to agents: adopt, update and verify

1

Adopt

"How do I build on this, or move to the current version?"

Generates the integration and handles migrations off deprecated versions

2

Update

"What changed that affects my flow?"

Brings new rules, like the data-handling one, to the teams they affect

3

Verify

"Is this still valid?"

Checks output against current rules, before launch and long after

A geo-expansion platform can expose one tool of each kind through a single MCP server. **The pattern fits any platform with rules to keep.**

SOURCE: speaker's platform; team analysis

When agents become your customers

Two agents from other systems that now ask the platform first

A design agent that asks the platform first keeps screens and production in step

BEFORE	WITH A DESIGN AGENT AS A CUSTOMER
A designer works in the home market and copies the layout for other markets	The design tool's agent asks the platform's MCP server which layouts and content vary by market
The platform renders each market correctly in production	Designs for each market show what production will render
QA compares production to the design files, finds mismatches, and nobody knows which one is right	QA compares like with like, and the designer never had to learn the market rules

People do not need to know every market rule; **their agents need to know where to ask**

SOURCE: speaker's platform design; e.g. a design tool's agent (an MCP client) calling the platform's MCP server

When a policy changes, an agent checks production and opens the fix for a person to approve

1

Policy changes

A policy is updated in the company's compliance system of record

→

2

Agent checks production

An agent asks the platform's MCP server for the live production metadata

→

3

Drift found

It compares the two and finds where production no longer matches the policy

→

4

Team notified

It tells the owning team and opens a pull request with the update

→

5

Person approves

A human reviews and approves before anything ships

BEFORE

Policy changes waited for a team to notice, or for an audit to find the gap

NOW

Every policy change triggers a production check, and the fix arrives as a reviewable pull request

The policy system becomes a customer of the platform, **and a person still approves every change**

SOURCE: speaker's platform design; system of record = the company's compliance policy database

THE COMPLIANCE GATE

Two loops are the road's guardrails: agents find and flag, people decide

LOOP 1 · TEAMS BUILDING ON THE PLATFORM

A team's agent sends the product requirements to the platform through the MCP server

↓

The platform surfaces every feature that needs market configuration, and every compliance rule that applies

↓

The team builds them in from day one

ONE SOURCE OF TRUTH

The platform's rule set

← read by teams updated by custodians →

LOOP 2 · PEOPLE WHO OWN THE RULES

Every month, and on demand, an auto-research loop scans the sources the rules depend on: standards bodies, government and regulatory sources

↓

It scores how well the rules still match, flags any drop below the threshold, and proposes the update

↓

A platform custodian validates and approves every change

Agents do the searching and flagging; **no rule changes without a person approving it**

SOURCE: speaker's platform design; auto-research loop inspired by Karpathy's autoresearch (Mar 2026)

THE COMPLIANCE GATE

Every answer traces back to a rule version that a person approved

1

Change flagged

The scan records which source changed, and when

→

2

Person approves

A custodian approves the update, with the reason

→

3

Version published

The rule set gets a new version, with an effective date

→

4

Answers stamped

Every answer is scoped to the team's market and names the version behind it

THE AUDITOR ASKS

"Why did the platform give this answer, on that date?"

THE PLATFORM ANSWERS

The rule version that applied, the source behind it, and the person who approved it

Human approval is what makes the trail credible: **each approval sits on the record next to the flag that triggered it**

SOURCE: speaker's platform design; approach only. Record-keeping expectations in financial services, e.g. EU DORA RTS 2024/1774 Art. 12

RESULTS

DIRECTIONAL

The impact shows up in three places, one for each part of the fix

FINDING THE RULES

At design

Requirements surface when the product requirements are written, so compliance stops arriving as an escalation

WHAT TO MEASURE

Share of requirements found before code; late compliance escalations per quarter

UPGRADES

Days → hours

Moving off a deprecated version becomes a platform job, so teams stop deferring it as tech debt

WHAT TO MEASURE

Effort per repository per upgrade; time for every consumer to reach the current version

KEEPING RULES CURRENT

Every month

Source changes are caught by the scan and approved by a custodian, long before an audit asks

WHAT TO MEASURE

Time from a source change to an approved platform update

50–80% less effort and fewer late surprises: the speaker's directional range for this pattern

SOURCE: speaker's platform experience; directional findings and the speaker's estimate for other teams

FOR YOUR PLATFORM

Five moves any platform team can copy to put its agents on the paved road

MOVE	QUESTION FOR YOUR TEAM
1. Find your late loop	Where do teams meet your platform's rules after the code is written?
2. Expose the platform where work starts	Can an agent reach your platform from the tools teams already use?
3. Offer adopt, update and verify tools	Can an agent build on the platform, hear what changed and check its output?
4. Make upgrades the platform's job	Is your next end-of-life a notice you send, or a migration your platform runs?
5. Keep people owning the rules	Do agents flag changes while a person approves every rule update?

Start with the rule that changed most recently.

SOURCE: team analysis

Make the road easy to find.
Keep people at the gate.

Agents find and flag · People decide

Sumeet Ram
linkedin.com/in/sumeetlram